Лабораторна робота 5

Навігація і використання ресурсів у React

Мета роботи

Навчитися створювати сторінки React-застосунку, переходити між ними за допомогою React Router, підключати зображення та відео, завантажувати ресурс за дією користувача й перевіряти числове введення. Використати TypeScript для опису стану та обробників.

Результат роботи

Один вебзастосунок на React + TypeScript із чотирма сторінками:

Адреса	Пункт меню	Вміст
`/`	Головна	Привітання
`/media`	Зображення і відео	Три зображення, зокрема одне з динамічним імпортом, та вбудоване відео
`/about`	Про програму	Призначення застосунку
`/calculator`	Калькулятор	Одне поле введення, накопичуваний результат і арифметичні дії

На всіх сторінках має бути спільне меню. Для невідомої адреси передбачте повідомлення «Сторінку не знайдено» та посилання на головну.

Це робота з **React для браузера**, а не з React Native. Серверна частина, база даних, реєстрація, Redux та збереження в localStorage не потрібні.

Демо для ознайомлення

Поруч є демо-проєкт «Стежка в парку». Він демонструє навігацію, способи підключення ресурсів, динамічний імпорт, умовне відображення та форму розрахунку часу прогулянки. У файлах є пояснювальні коментарі. Демо має іншу тему й не містить калькулятора з умови роботи.

Спочатку запустіть демо за його інструкцією, перегляньте поведінку сторінок, а потім створіть **власний** проєкт за кроками нижче.

1. Підготовка проєкту

Потрібні редактор коду, браузер, Node.js підтримуваної LTS-версії, сумісної з Vite, і npm. Для цієї роботи підійде Node.js 24 LTS. Перевірте встановлення в терміналі:

```
node --version
npm --version
```

Node.js запускає інструменти розробки на комп'ютері. Сам інтерфейс працюватиме у браузері. npm встановлює пакети та запускає команди з `package.json`.

У папці, де зберігаєте лабораторні роботи, виконайте:

```
npm create vite@latest lab5-navigation -- --template react-ts
cd lab5-navigation
npm install
npm install react-router
npm run dev
```

`react-ts` — шаблон React із TypeScript; `react-router` — бібліотека маршрутизації. Якщо npm запропонує встановити `create-vite`, погодьтеся. Якщо майстер запропонує самостійно встановити залежності й запустити сервер, можна відмовитися та виконати наведені команди вручну. Якщо папка вже існує, виберіть іншу назву: не погоджуйтеся на видалення своїх файлів.

Відкрийте адресу біля `Local` у терміналі. Поки працює `npm run dev`, сервер доступний у браузері; `Ctrl+C` його зупиняє. Зміни у файлах після збереження зазвичай відразу з'являються на сторінці.

У цій роботі використовується декларативний режим React Router: маршрути описуються компонентами. Встановлюємо `react-router` та імпортуємо його компоненти саме з `react-router`. Не потрібно додатково встановлювати `@types/react-router`: пакет уже містить типи. Не змішуйте приклади різних версій із `Switch`, `component={...}` або `exact` із наведеним нижче синтаксисом.

Орієнтовна структура:

```
lab5-navigation/
├── public/                # Файли, які доступні за URL без імпорту
├── src/
│   ├── assets/
│   │   ├── campus.jpg     # Локальне зображення
│   │   └── detail.jpg     # Зображення для динамічного імпорту
│   ├── components/
│   │   └── Navigation.tsx
│   ├── pages/
│   │   ├── HomePage.tsx
│   │   ├── MediaPage.tsx
│   │   ├── AboutPage.tsx
│   │   ├── CalculatorPage.tsx
│   │   └── NotFoundPage.tsx
│   ├── App.tsx
│   ├── main.tsx
│   └── index.css
├── index.html
├── package.json
└── ...                    # Конфігурація, створена Vite
```

Назви й розширення зображень мають відповідати вашим реальним файлам. `.tsx` використовують для TypeScript із JSX, `.ts` — для коду без JSX. Не запускайте `App.tsx` через `node`: Vite підготує його для браузера.

2. Короткі теоретичні відомості

2.1. Що таке сторінка й маршрут

У цій роботі сторінка — звичайний React-компонент. **Маршрут** пов'язує адресу з компонентом: наприклад, `/about` → `AboutPage`.

Браузер спочатку завантажує HTML і код застосунку. Далі React Router може змінювати адресу та показувати потрібний компонент без повного перезавантаження документа. Такий підхід називають клієнтською маршрутизацією. Кнопки браузера «Назад» і «Вперед» продовжують працювати.

Основні компоненти:

- `BrowserRouter` надає застосунку доступ до поточної адреси й історії браузера. Для нашої структури потрібна одна така обгортка.
- `Routes` вибирає відповідний маршрут серед вкладених `Route`.
- `Route` задає `path` — шлях і `element` — JSX, який треба показати.
- `Link` створює внутрішнє посилання без повного перезавантаження сторінки.
- `NavLink` додатково визначає, чи є посилання активним; це зручно для меню.

Для внутрішньої сторінки використовуйте `<Link to="/about">Про програму</Link>`, для зовнішнього сайту чи завантаження файла — звичайний `<a href="...">`.

2.2. URL — не шлях до файла на комп'ютері

`/media` — адреса сторінки застосунку, а не файл `media.html`. `../assets/campus.jpg` в імпорті — шлях від поточного файла коду до ресурсу. `/campus.jpg` у `src` зображення — адреса ресурсу від кореня сайту. Шляхи на кшталт `C:\Pictures\photo.jpg` або `/Users/name/photo.jpg` не дозволять іншим користувачам побачити ваше зображення.

2.3. Статичний і динамічний імпорт

Статичний імпорт записують на верхньому рівні файла:

```
import campusUrl from '../assets/campus.jpg';
```

Vite обробляє файл і надає значення `campusUrl`: URL, який можна передати в `<img src={campusUrl}>`. Це не React-компонент і не самі пікселі зображення. Під час збірки Vite може змінити ім'я ресурсу або вбудувати маленький файл у URL даних.

Динамічний імпорт — виклик `import(...)`, який виконується під час роботи програми:

```
const imageModule = await import('../assets/detail.jpg');
const imageUrl = imageModule.default;
```

Він повертає `Promise` — результат, який стане доступним пізніше. `await` чекає на нього всередині `async`-функції, не блокуючи весь інтерфейс. Для імпортованого зображення `default` містить URL. Щоб показати його, URL потрібно записати в стан компонента.

Важлива різниця: `{isVisible && <img src={campusUrl} />}` лише керує появою елемента. Якщо `campusUrl` отримано статичним імпортом, це **не динамічний імпорт**. Також `loading="lazy"` на `<img>` — окремий механізм браузера, а не заміна `import()`.

У Vite використовуйте `import` / `export`, а не CommonJS `require`. Для навчального прикладу вказуйте в `import()` конкретний рядок шляху, який Vite може знайти під час збірки.

2.4. Що тут дає TypeScript

```
const [imageUrl, setImageUrl] = useState<string | null>(null);
const [result, setResult] = useState(0);
const [input, setInput] = useState('');
```

`string | null` означає «рядок або відсутність значення». Перевірка `imageUrl !== null` дозволяє використати URL лише тоді, коли він уже є. Типи `result` і `input` визначаються автоматично за початковими значеннями: число та рядок відповідно.

TypeScript допомагає помітити неправильні типи під час розробки, але не перевіряє доступність зовнішнього відео, правильність введеного числа чи можливість ділення на нуль. Ці перевірки потрібно написати самостійно.

3. Завдання 1. Сторінки, навігація та ресурси

Умова

Створіть сторінки «Головна», «Зображення і відео», «Про програму». На головній покажіть «Вітаємо на Головній сторінці!», на сторінці про програму — «Цей додаток демонструє навігацію та роботу з ресурсами в React».

На сторінці медіа розмістіть:

1. Локальне зображення зі статичним імпортом із `src/assets` або за URL із `public`.
2. Зображення із зовнішнього джерела за прямим HTTPS-посиланням.
3. Інше локальне зображення, URL якого отримується через `import()` **після натискання кнопки**.
4. Відео, вбудоване через `iframe`, із посиланням для відкриття на сайті джерела.

Використовуйте власні або дозволені до використання зображення. Під зовнішніми матеріалами зазначте джерело. Оформлення просте й довільне; важливі читабельність та правильна робота.

Крок 1. Створіть компоненти сторінок

У `src/pages/HomePage.tsx`:

```
export default function HomePage() {
  return <h1>Вітаємо на Головній сторінці!</h1>;
}
```

За аналогією створіть `AboutPage.tsx` із заголовком і потрібним абзацом, а також `MediaPage.tsx` із заголовком «Зображення і відео». Поки що цього достатньо, щоб перевірити маршрути. У `NotFoundPage.tsx` додайте повідомлення та `Link` на `/`; не забудьте імпортувати `Link` із `react-router`.

Назва компонента починається з великої літери. `export default` дозволяє імпортувати його без фігурних дужок. Кожен компонент повертає JSX; кілька сусідніх елементів загорніть, наприклад, у `<section>`.

Крок 2. Підключіть маршрутизатор

У `src/main.tsx` залиште створення кореня React, але обгорніть `App` у `BrowserRouter`:

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router';
import App from './App';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
);
```

`root` — елемент у `index.html`, у якому React показує інтерфейс. Символ `!` повідомляє TypeScript, що цей елемент існує; він не створює елемент і не є перевіркою під час виконання. У шаблоні Vite відповідний `<div id="root">` уже є. `StrictMode` допомагає виявляти проблеми під час розробки.

Не додавайте другий `BrowserRouter` в `App` або в окремі сторінки.

Крок 3. Зробіть спільне меню

У `src/components/Navigation.tsx`:

```
import { NavLink } from 'react-router';

export default function Navigation() {
  return (
    <nav aria-label="Основна навігація">
```

```
      <NavLink to="/" end>Головна</NavLink>
      <NavLink to="/media">Зображення і відео</NavLink>
      <NavLink to="/about">Про програму</NavLink>
    </nav>
  );
}
```

`to` містить адресу переходу. `NavLink` надає активному посиланню клас `active` і позначає його як поточну сторінку для допоміжних технологій. `end` задає зіставлення до кінця шляху; для кореневого `/` React Router також має спеціальну поведінку, щоб не позначати його активним на кожній сторінці.

Мінімальні стилі в `src/index.css`:

```
body { margin: 0; font-family: Arial, sans-serif; color: #172033; }
#root { max-width: 1000px; margin: 0 auto; padding: 24px; }
nav { display: flex; flex-wrap: wrap; gap: 16px; margin-bottom: 24px; }
nav a { color: #1554b5; padding: 8px 0; }
nav a.active { font-weight: bold; text-decoration-thickness: 3px; }
img { display: block; max-width: 100%; height: auto; }
button, input { font: inherit; }
button { cursor: pointer; }
:focus-visible { outline: 3px solid #1554b5; outline-offset: 4px; }
.video-frame { width: 100%; aspect-ratio: 16 / 9; border: 0; }
```

Приберіть або замініть стартові стилі Vite, якщо вони обмежують ширину, центрують усе по вертикалі чи задають небажану темну тему. Перевірте, чи `App.tsx` ще імпортує `App.css`.

Крок 4. Зв'яжіть адреси зі сторінками

У `src/App.tsx`:

```
import { Route, Routes } from 'react-router';
import Navigation from './components/Navigation';
import HomePage from './pages/HomePage';
import MediaPage from './pages/MediaPage';
import AboutPage from './pages/AboutPage';
import NotFoundPage from './pages/NotFoundPage';

export default function App() {
  return (
    <>
      <Navigation />
      <main>
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/media" element={<MediaPage />} />
          <Route path="/about" element={<AboutPage />} />
          <Route path="*" element={<NotFoundPage />} />
        </Routes>
      </main>
    </>
```

```
  );
}
```

Меню знаходиться поза `Routes`, тому залишається видимим незалежно від вибраної сторінки. `element` отримує JSX `<HomePage />`, а не просто назву функції. `*` відповідає адресам, для яких немає конкретного маршруту. `<>...</>` — фрагмент, який групує елементи без зайвого HTML-контейнера.

Спочатку перевірте всі переходи й кнопку «Назад», а вже потім додавайте медіа.

Крок 5. Додайте локальне й зовнішнє зображення

Скопіюйте власний файл у `src/assets/campus.jpg`. На початку `MediaPage.tsx` додайте імпорт, а в JSX — зображення:

```
import campusUrl from '../assets/campus.jpg';

// Усередині JSX компонента:
<figure>
  <img src={campusUrl} alt="Головний вхід до навчального корпусу" />
  <figcaption>Локальний файл із src/assets.</figcaption>
</figure>
```

Це фрагмент: рядок імпорту розміщується перед функцією, а `<figure>` — у її `return`. `alt` коротко описує зміст зображення, якщо воно не завантажилося або сторінку читають за допомогою скринрідера. Підпис повинен відповідати саме вашому файлу.

Альтернатива: покладіть зображення в `public/campus.jpg` і використайте:

```
<img
  src={`${import.meta.env.BASE_URL}campus.jpg`}
  alt="Головний вхід до навчального корпусу"
/>
```

Папку `public` у URL не пишуть. `BASE_URL` — базовий шлях застосунку, який надає Vite; у звичайному локальному запуску це `/`. Такі файли копіюються у збірку без перейменування. Для першого зображення достатньо **одного** із цих способів.

Для другого зображення вкажіть пряму HTTPS-адресу файла у `src`. Посилання на сторінку пошуку зображень не підходить. Адреса не обов'язково закінчується на `.jpg`, але у відповіді має бути саме зображення. Перевірте її в браузері. Зовнішній сервер може заборонити вбудовування або стати недоступним; це не залежить від TypeScript.

Щоб показати зрозуміле повідомлення при помилці, додайте `useState(false)` для ознаки помилки й установлюйте `true` в `onError` зображення. За цією ознакою показуйте або `<img>`, або текст із посиланням на джерело. Робочий приклад такого підходу є в `ResourcesPage.tsx` демо.

Крок 6. Завантажте третє зображення після натискання

Покладіть інший файл у `src/assets/detail.jpg`. Створіть `src/components/ExtraImage.tsx`:

```
import { useState } from 'react';

export default function ExtraImage() {
  const [url, setUrl] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState('');

  async function handleLoad() {
    setIsLoading(true);
    setError('');

    try {
      const imageModule = await import('../assets/detail.jpg');
      setUrl(imageModule.default);
    } catch {
      setError('Не вдалося отримати ресурс. Спробуйте ще раз.');
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <section>
      <h2>Додаткове зображення</h2>
      <button
        type="button"
        onClick={handleLoad}
        disabled={isLoading || url !== null}
      >
        {isLoading ? 'Завантаження…' : 'Показати зображення'}
      </button>
      {error && <p role="alert">{error}</p>}
      {url !== null && (
        <img
          src={url}
          alt="Деталь навчального корпусу"
          onError={() => {
            setUrl(null);
            setError('Файл отримано, але зображення не відобразилося.');
          }}
        />
      )}
    </section>
  );
}
```

Імпортуйте `ExtraImage` у `MediaPage.tsx` і додайте `<ExtraImage />` до JSX. За потреби змініть назву файла й `alt`.

Послідовність роботи:

1. Спочатку `url` дорівнює `null`, тому `<img>` ще немає в інтерфейсі.
2. Натискання викликає `handleLoad`. Кнопка тимчасово вимикається через `disabled`, щоб не запускати повторні завантаження.
3. `import()` отримує модуль ресурсу, а `setUrl` записує URL у стан. React повторно обчислює JSX, і браузер починає показувати зображення.
4. `catch` обробляє помилку імпорту, `finally` завершує стан очікування незалежно від успіху. Помилка завантаження самого зображення обробляється окремо через `onError`.

`isLoading` тут описує очікування **імпорту**, а не повне завантаження пікселів. Повторні звернення можуть використовувати кеш. Маленькі ресурси Vite може вбудувати у код, тому окремий HTTP-запит до `.jpg` не гарантований. Це не змінює того, що виклик `import()` відбувається після натискання.

Крок 7. Вбудуйте відео

Знайдіть доступне для вбудовування відео. Для YouTube відкрийте «Поділитися» → «Вставити» й отримайте адресу виду `https://www.youtube.com/embed/VIDEO_ID`. Адреса звичайної сторінки `watch?v=...` не є адресою вбудованого програвача.

У `MediaPage.tsx` додайте:

```
<iframe
  className="video-frame"
  src="https://www.youtube.com/embed/VIDEO_ID"
  title="Відео про обрану тему"
  loading="lazy"
  allow="encrypted-media; picture-in-picture; fullscreen"
  allowFullScreen
/>
<p>
  <a
    href="https://www.youtube.com/watch?v=VIDEO_ID"
    target="_blank"
    rel="noopener noreferrer"
  >
    Відкрити відео на YouTube
  </a>
</p>
```

Замініть `VIDEO_ID` в обох місцях на справжній ідентифікатор. `title` пояснює зміст фрейма, `loading="lazy"` дозволяє браузеру відкласти його завантаження, а CSS з `aspect-ratio` зберігає пропорції. У JSX використовуємо `allowFullScreen`, а рамку прибираємо стилем `border: 0`.

Власник може заборонити вбудовування; браузерні розширення, політика мережі або відсутність інтернету також можуть блокувати програвач. Не намагайтеся обійти обмеження: оберіть інше дозволене відео. Посилання на сторінку джерела залишайте в будь-якому разі. Не вмикайте автоматичне відтворення.

Самоперевірка завдання 1

- Кожен пункт меню змінює URL та відповідний вміст; активний пункт помітний.
- «Назад» / «Вперед» працюють; після оновлення `/media` у локальному Vite сторінка відкривається.
- Адреса `/unknown` показує повідомлення про відсутню сторінку, а меню залишається.
- Є три різні зображення; до натискання кнопки третього зображення немає.
- У коді третього зображення використано саме `await import(...)`, а не `require` чи тільки умовний JSX.
- Зображення мають змістовний `alt`; зовнішні ресурси мають підпис із джерелом.
- Відео можна переглянути або відкрити за резервним посиланням.
- На вузькому екрані медіа не виходять за межі сторінки; до меню й кнопок можна перейти клавішею Tab.

4. Завдання 2. Калькулятор із накопичуваним результатом

Умова

Додайте до цього самого застосунку сторінку `/calculator` і пункт меню «Калькулятор». На сторінці мають бути:

- поточний результат, початково `0`;
- одне поле для введення числа;
- кнопки «Додати», «Відняти», «Помножити», «Поділити»;
- окремі кнопки «Очистити поле» та «Скинути результат»;
- зрозуміле повідомлення про некоректне введення або ділення на нуль.

Кожна дія застосовує введене число до **поточного результату**. Наприклад: результат `0`, ввели `8` і натиснули «Додати» → `8`; замінили поле на `3` і натиснули «Помножити» → `24`. Це не калькулятор текстових виразів: вводити `8 * 3` не потрібно. `eval` не використовуйте.

Після успішної операції поле зберігає введене значення. «Очистити поле» не змінює результат; «Скинути результат» установлює `0`, але не змінює поле. Обидві кнопки прибирають повідомлення про помилку. Некоректна операція не повинна змінювати попередній результат.

Крок 1. Підключіть сторінку

Створіть `src/pages/CalculatorPage.tsx`, імпортуйте компонент в `App.tsx`, додайте `<Route path="/calculator" element={<CalculatorPage />} />` до `Routes` і відповідний `NavLink` у `Navigation.tsx`.

Крок 2. Опишіть стан і поле

У компоненті потрібні три значення:

```
const [input, setInput] = useState('');
const [result, setResult] = useState(0);
const [error, setError] = useState('');
```

Не забудьте `import { useState } from 'react'`. Поле залишається рядком: користувач може його очистити або ще не завершити введення. Навіть `input` із `type="number"` надає `event.currentTarget.value` як рядок.

```
<label htmlFor="operand">Число для операції</label>
<input
  id="operand"
  type="number"
  step="any"
  value={input}
  onChange={event => {
    setInput(event.currentTarget.value);
    setError('');
  }}
/>
<p aria-live="polite">Результат: {result}</p>
{error && <p role="alert">{error}</p>}
```

`htmlFor` пов'язує підпис із полем через `id`. `step="any"` допускає дробові значення. TypeScript визначає тип `event` для такого обробника в JSX автоматично. Якщо ви винесете обробник в окрему функцію, можна використати `import type { ChangeEvent } from 'react'` і параметр `event: ChangeEvent<HTMLInputElement>`.

`aria-live="polite"` допомагає оголосити зміну результату користувачам скринрідера. Нуль — повноцінний результат, тому виводьте `{result}` без умови `result && ...`.

Крок 3. Перевіряйте введення перед операцією

У кожному обробнику:

1. Якщо `input.trim() === ''`, покажіть повідомлення «Введіть число» й завершіть обробник через `return`.
2. Перетворіть рядок на число: `const operand = Number(input)`.
3. Якщо `!Number.isFinite(operand)`, повідомте про некоректне число та завершіть обробник.
4. Для ділення додатково перевірте `operand === 0`.
5. Обчисліть наступний результат і перевірте `Number.isFinite(nextResult)`. Надто великі значення можуть дати `Infinity`; їх не записуйте в стан.
6. Якщо все правильно, оновіть результат і приберіть попередню помилку.

Порожнє значення перевіряють **до** `Number`, тому що `Number('')` дорівнює `0`. Перевірка `if (!operand)` не підходить: вона помилково відхилить коректний нуль. Не покладайтеся тільки на вигляд поля: перевірка потрібна й у коді.

Приклад одного обробника:

```
function handleAdd() {
  if (input.trim() === '') {
    setError('Введіть число.');
    return;
  }

  const operand = Number(input);
  if (!Number.isFinite(operand)) {
    setError('Введіть скінченне число.');
    return;
  }

  const nextResult = result + operand;
  if (!Number.isFinite(nextResult)) {
    setError('Результат завеликий для обчислення.');
    return;
  }

  setResult(nextResult);
  setError('');
}
```

Обробник отримує `result` із поточного рендеру й один раз оновлює його. Для послідовних окремих натискань цього достатньо. Якщо використовуєте функціональне оновлення `setResult(previous => ...)`, залишайте його чистим: не викликайте `setError` усередині функції обчислення нового стану.

Крок 4. Реалізуйте інші дії та скидання

Самостійно додайте віднімання, множення й ділення за тією самою схемою. Оператори JavaScript: `+`, `-`, `*`, `/`. Перевірка нульового дільника має виконуватися до ділення; навіть `0 / 0` є помилкою для цього калькулятора.

Приєднуйте функції до кнопок, не викликаючи їх під час рендеру:

```
<button type="button" onClick={handleAdd}>Додати</button>
```

Для очищення поля викличте `setInput('')` та `setError('')`; для скидання результату — `setResult(0)` та `setError('')`. Це різні дії, тому одна не повинна неочікувано виконувати іншу.

За бажанням винесіть спільну перевірку введення в допоміжну функцію. Не ускладнюйте компонент парсингом виразів, клавіатурою калькулятора чи глобальним сховищем.

Числа JavaScript використовують двійкове представлення: наприклад, `0.1 + 0.2` може дати `0.30000000000000004`. Це не помилка React. Можна форматувати відображення через `Intl.NumberFormat`, але не замінюйте числовий стан рядком і не округляйте приховано після кожного кроку.

Самоперевірка завдання 2

Виконайте послідовно:

Дія	Очікування
Початковий стан	Результат `0`, поле порожнє
Ввести `8`, «Додати»	`8`
Замінити поле на `3`, «Помножити»	`24`
Замінити поле на `4`, «Відняти»	`20`
Замінити поле на `2`, «Поділити»	`10`
Замінити поле на `0`, «Поділити»	Повідомлення про помилку; результат залишається `10`
З тим самим `0`, «Помножити»	Результат `0`, повідомлення прибрано
Ввести `-2.5`, «Додати»	Результат `-2.5`
«Очистити поле»	Поле порожнє, результат `-2.5`
З порожнім полем, «Додати»	Помилка, результат не змінено
Ввести `7`, «Скинути результат»	Результат `0`, у полі залишається `7`

Окремо перевірте дуже великі числа: у результаті не повинні з'являтися `Infinity` або `NaN`.

При переході на іншу сторінку React прибирає компонент калькулятора з дерева. Якщо стан зберігається всередині `CalculatorPage`, після повернення він почнеться з початкових значень. Для цієї роботи це очікувана поведінка; збереження між сторінками не вимагається.

5. Завершення та перевірка

У папці власного проєкту виконайте:

```
npm run build
npm run lint
npm run preview
```

У стандартному шаблоні React + TypeScript команда `build` перевіряє типи й створює файли в `dist`. `lint` перевіряє правила коду. `preview` локально показує готову збірку; відкрийте вказану ним адресу та повторіть основні перевірки. Це не сервер для постійної публікації сайту.

Перевірте вікно Console в інструментах розробника браузера: не має бути необроблених помилок застосунку. У Network можна побачити запити до зображень та імпортованих модулів. Кеш може приховувати повторне завантаження, тому для спостереження відкрийте сторінку наново з вимкненим кешем. Не кожен ресурс обов'язково буде окремим файлом після оптимізації.

Типові проблеми

Симптом	Що перевірити
Помилка, що посилання має бути всередині Router	`BrowserRouter` повинен обгортати весь `App` у `main.tsx`
Помилка про вкладений Router	Залиште лише одну обгортку `BrowserRouter`
Компонент або зображення не знайдено	Шлях, регістр і розширення файла; правильний `export` та `import`
Зображення з `public` не відкривається	У URL не повинно бути слова `public`; перевірте `BASE_URL`
Зовнішнє зображення не показується	Прямий HTTPS-URL, доступність джерела та дозвіл на вбудовування
Відео не працює	Адреса `/embed/…`, правильний ID, інтернет і дозвіл власника
Натискання кнопки перезавантажує сторінку	У формі кнопки операцій повинні мати `type="button"`; для надсилання форми використовуйте `preventDefault()`
Замість суми виходить `23`	Рядки `'2'` та `'3'` додано без перетворення на числа
Нуль зникає або відхиляється	Не перевіряйте число тільки через його істинність

Після публікації `BrowserRouter` потребує налаштування хостингу: запит сторінки на кшталт `/media` має повертати `index.html`, а реальні ресурси — свої файли. Інакше пряме відкриття чи оновлення вкладеної адреси може дати серверну 404. Локальний сервер Vite це обслуговує автоматично. Публікація не є обов'язковою частиною роботи.

Якщо пізніше розміщуватимете сайт у підпапці, узгодьте `base` у Vite та `basename` у `BrowserRouter`. Для запуску цієї роботи локально їх змінювати не потрібно.

Підготуйте вихідні файли з `package.json` і `package-lock.json`, короткою інструкцією запуску та знімками основних сторінок. Не додавайте в архів `node_modules` і `dist`: їх можна відновити командами встановлення та збірки. За наявності узгодженого `package-lock.json` команда `npm ci` встановлює зафіксовані версії залежностей.

6. Захист роботи

Захист відбувається усно з демонстрацією застосунку. Покажіть усі маршрути, активне меню, роботу «Назад» / «Вперед», три способи отримання зображень, відео та всі дії калькулятора. Продемонструйте порожнє введення, ділення на нуль і незалежне очищення поля та результату.

Поясніть структуру програми: де оголошено маршрути, чому меню спільне, де зберігається стан, коли виконується `import()` і що відбувається при помилці зовнішнього ресурсу.

Будьте готові внести невелику зміну: додати просту сторінку й пункт меню, замінити ресурс, змінити текст повідомлення, додати кнопку зміни знака результату або скоригувати формат його відображення. Поясніть, які файли й частини логіки це зачіпає.

Контрольні питання

1. Чим маршрут відрізняється від компонента сторінки?
2. Для чого потрібні `BrowserRouter`, `Routes` і `Route`?
3. Коли використовують `Link`, `NavLink` та звичайний `<a>`?
4. Чому спільне меню розміщене поза `Routes`?
5. Чим відрізняються ресурси в `src/assets` і `public`?
6. Що повертає `import()` і що містить `default` при імпорті зображення у Vite?
7. Чому умовний показ `<img>` не дорівнює динамічному імпорту?
8. Для чого потрібні `alt` зображення та `title` фрейма?
9. Чому число в полі зберігають як рядок і чому порожнє поле перевіряють перед `Number`?
10. Які перевірки потрібні для ділення? Чому TypeScript не виконує їх автоматично?
11. Чим очищення поля відрізняється від скидання результату?
12. Чому після виходу зі сторінки локальний стан може зникнути? Що потрібно хостингу для прямого відкриття маршруту?

7. Корисні джерела

Наведених пояснень і демо достатньо для початку роботи. Для уточнення окремих можливостей:

- Vite — створення проєкту.
- React Router — установлення в декларативному режимі.
- React Router — маршрути.
- React Router — навігація.
- Vite — робота зі статичними ресурсами.
- MDN — динамічний import().
- React — useState та керовані поля.
- React — TypeScript.
- MDN — iframe.
- MDN — Number.isFinite.